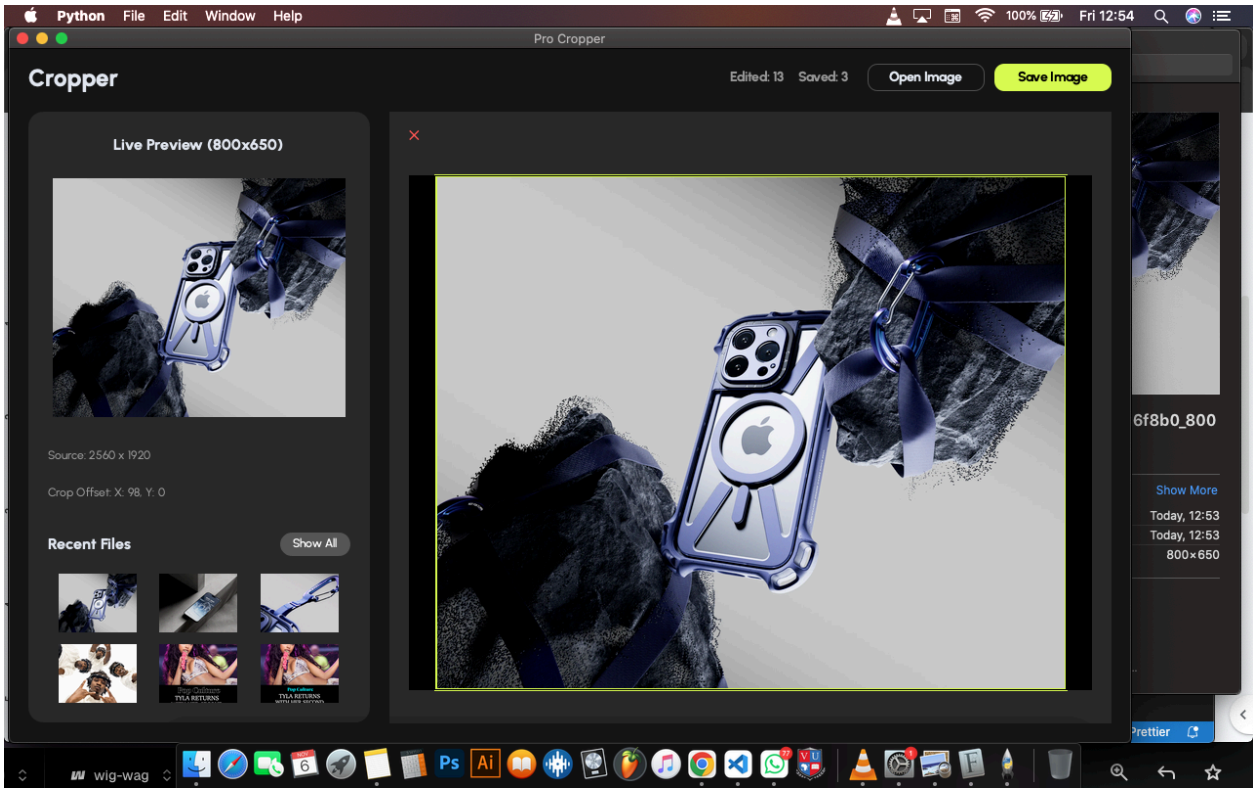
**Busuulwa Peter Nsereko - VU-BSF-2311-0200-DAY**

# PROJECT REPORT: DESIGN AND DEVELOPMENT OF CROPPER DESKTOP APPLICATION WITH PYTHON

**A Software Engineering Project Report**

---




## ABSTRACT

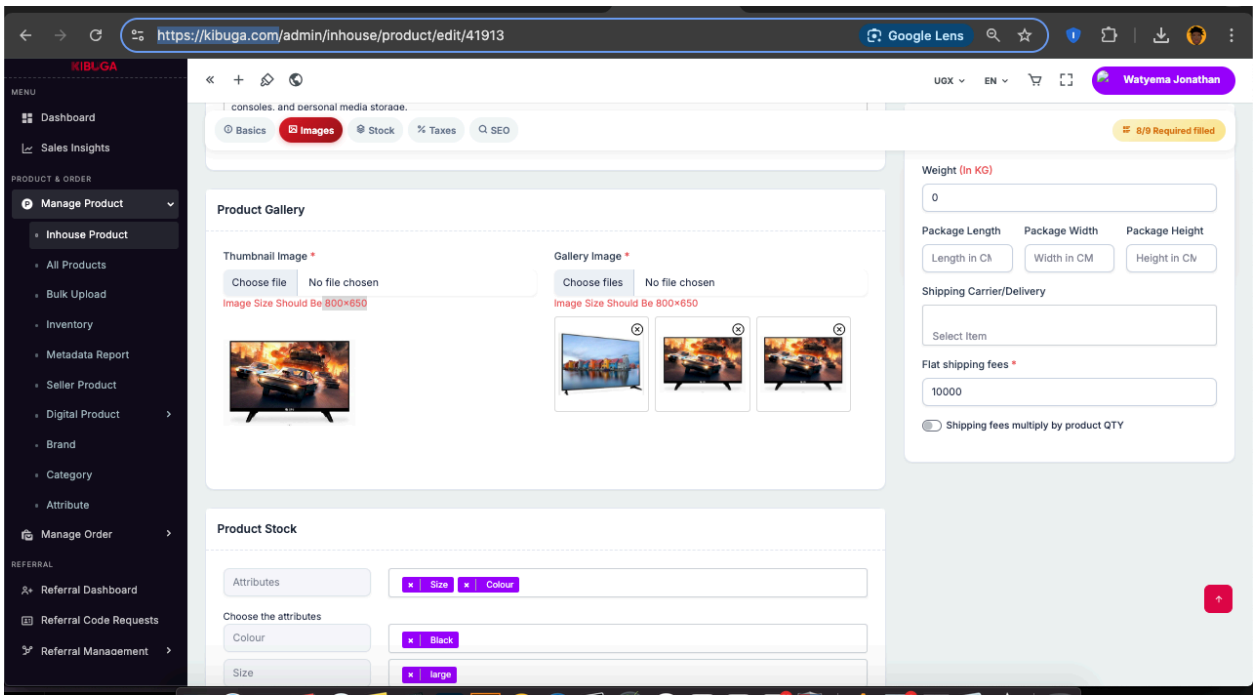
This project involved the design and development of Pro Cropper, a Python-based desktop application developed to simplify image cropping, resizing and format conversion. The application was designed to provide users with a consistent image output dimension of 800 × 650 pixels while maintaining the original image proportions and preventing distortion as test case for https://kibuga.com/, where images sumited via the admin dashboard sometimes have

Different dimensions and need editing.

The project followed the Software Development Life Cycle (SDLC), covering requirements identification, system design, implementation, testing and maintenance planning. Figma was used during the interface design phase to plan the application's layout, visual hierarchy and positioning of interactive components before implementation.

The application was developed using Python 3, CustomTkinter and the Pillow image-processing library. Its functionality includes image selection, interactive cropping, live preview, image format conversion, recent-file management, gallery navigation and session statistics. JSON-based storage was also incorporated to maintain application history, including recently accessed image paths and user activity statistics.

The resulting application demonstrates the practical application of software engineering principles, particularly user-centred interface design, modular development, image-processing algorithms, state management and software testing.




# CHAPTER ONE: INTRODUCTION

## 1.1 Background of the Project

Image processing is an important part of modern digital workflows, particularly in graphic design, digital publishing, content creation and online media management. Different platforms and publishing environments often require images to conform to specific dimensions and aspect ratios.

Manually resizing and cropping images can be time-consuming, especially when multiple images must maintain consistent dimensions. Conventional resizing methods may also distort images by stretching or compressing their original proportions.

Pro Cropper was developed to address these challenges by providing a dedicated desktop application that automatically maintains a standard output dimension while allowing users to control the visible portion of an image.

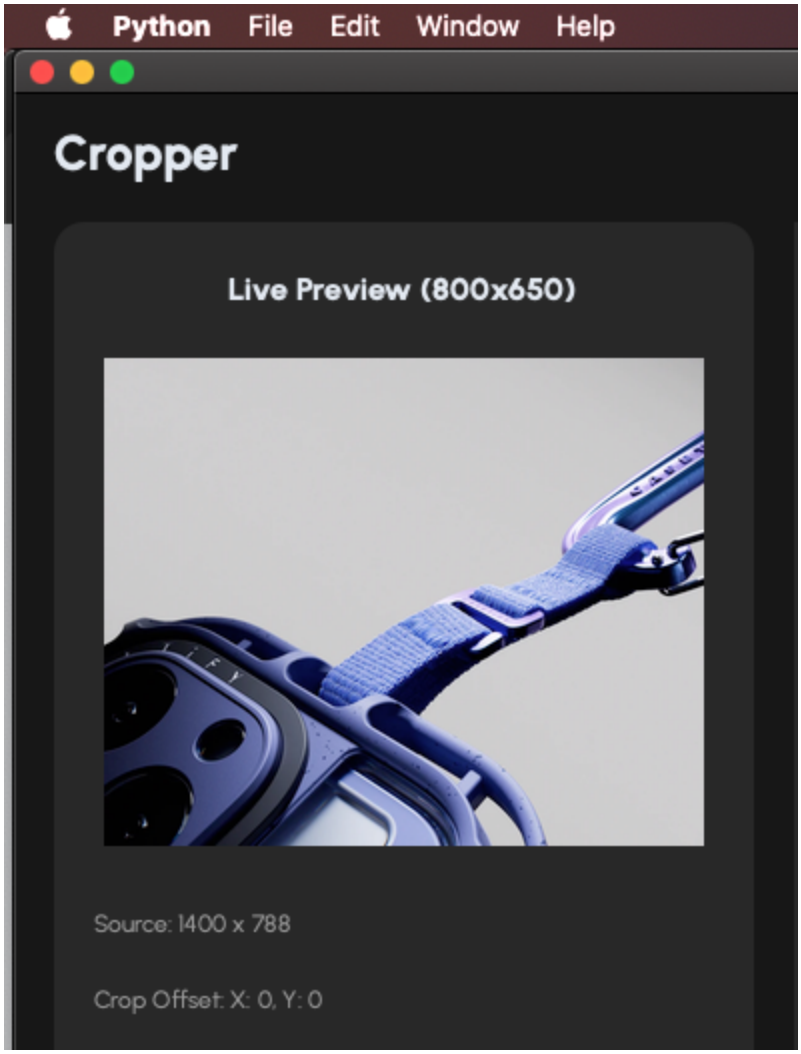
The application combines image-processing functionality with a graphical user interface designed to provide a straightforward and efficient editing experience.

## 1.2 Problem Statement

The project was motivated by the need for a simple image-processing tool capable of producing images with consistent dimensions without compromising their original proportions.

Existing manual workflows can require several steps to open, crop, resize and export images. Additionally, managing recently edited files and maintaining consistency across multiple outputs can become inefficient.

Pro Cropper addresses these challenges through an integrated desktop solution that combines image cropping, previewing, conversion and file management within one interface.




## 1.3 Project Objectives

**General Objective**

To design and develop a lightweight desktop image-processing application that enables users to crop and export images into a standard 800 × 650 pixel format without distortion.

**Specific Objectives**

1. To design an intuitive graphical user interface using Figma before implementation.
2. To develop an image-processing engine capable of maintaining a consistent 800:650 aspect ratio.
3. To implement interactive image positioning through mouse-based controls.
4. To support multiple image formats, including JPEG, PNG and WebP.
5. To provide a live preview of the selected cropping area.
6. To implement recent-file management and a gallery interface.

7. To maintain application statistics and recent-file information using JSON-based storage.
8. To test the application for functionality, reliability and correct image output.

### 1.4 Project Scope

The project focused on developing a desktop application with the following capabilities:

1. Opening and processing supported image formats.
2. Cropping images to an 800 × 650 output dimension.
3. Maintaining the original image proportions.
4. Interactive image positioning.
5. Live image preview.
6. Exporting cropped images in supported formats.
7. Displaying recently opened images.
8. Managing recent files through a gallery interface.
9. Tracking edited and saved image statistics.
10. Maintaining application history between sessions.

The application was developed primarily for desktop environments using Python.

---

# CHAPTER TWO: SYSTEM ANALYSIS AND DESIGN

## 2.1 Requirements Analysis

The requirements were identified based on the intended functionality and user experience of the application.

**Functional Requirements**

| Requirement | Description |
|---|---|
| Image Selection | Allow users to browse and open image files. |
| Image Cropping | Crop images to an 800 × 650 output dimension. |
| Image Positioning | Allow users to reposition images within the cropping area. |
| Live Preview | Display the resulting crop before saving. |
| Image Export | Save processed images in supported formats. |
| Recent Files | Display recently opened images. |

| | |
|---|---|
| Gallery Management | Provide access to the complete recent-file collection. |
| Statistics | Track the number of edited and saved images. |
| History Management | Maintain application history using JSON storage. |
| Session Management | Prompt users about unsaved changes before closing an active session. |

**Non-Functional Requirements**

- **Usability:** The interface should be intuitive and easy to navigate.
- **Performance:** Image processing should be efficient and responsive at 0.1sec.
- **Reliability:** The application should handle missing files without crashing.
- **Maintainability:** The code should be organised to support future modifications.
- **Consistency:** All exported images should conform to the required output dimensions.
- **Visual Consistency:** Interface components should follow a consistent design system.

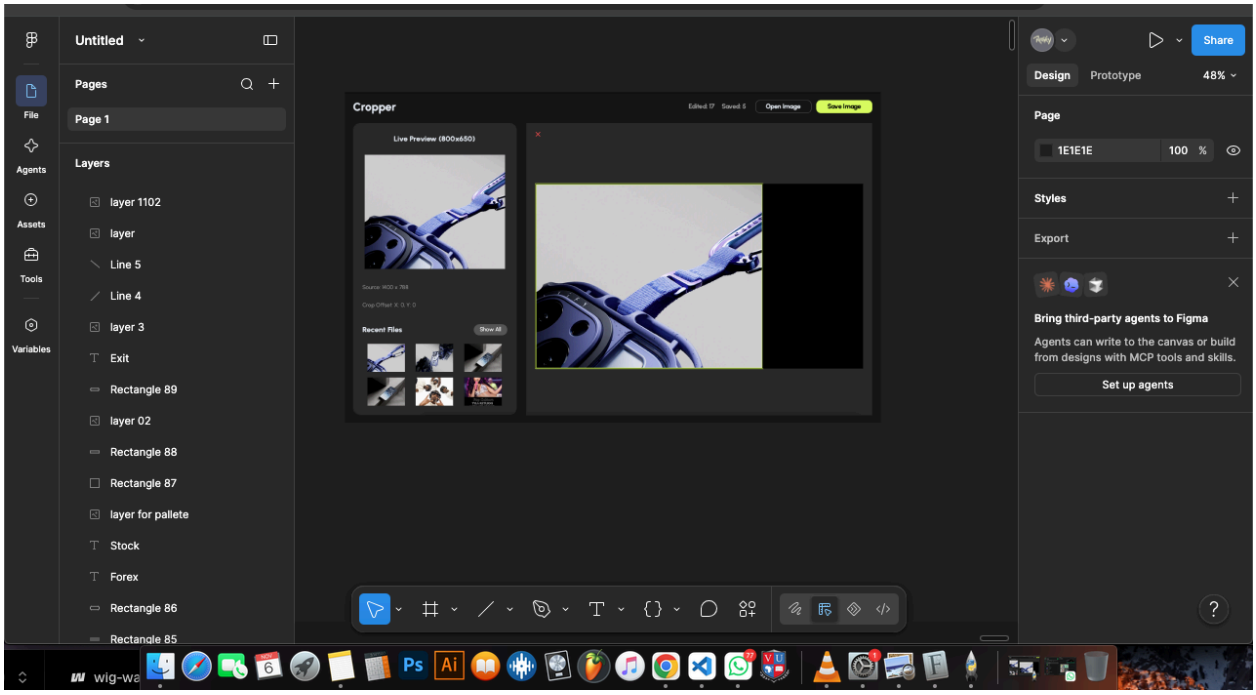
## 2.2 User Interface Design Using Figma

Before implementing the application in Python, Figma was used to plan and refine the graphical user interface.

This phase was important because it allowed the interface layout, visual hierarchy, component positioning and navigation experience to be considered before development.

The design adopted a modern dark-mode appearance, with contrasting colours and clearly positioned interactive controls.

The interface was divided into three main areas.




### A. Top Navigation Bar

The top navigation bar contains the primary application actions and activity statistics.

The following components were positioned in this section:

- **Open Image:** Positioned towards the top-right to allow users to select images.
- **Save Image:** Positioned beside the Open Image button and highlighted using a lime-green accent colour.
- **Edited Counter:** Displays the number of images opened for editing.
- **Saved Counter:** Displays the number of successfully exported images.

The statistics were positioned immediately before the primary action buttons to maintain a clean and organised navigation structure.

### B. Left Panel – Live Preview and Recent Files

The left panel was designed to provide a preview of the resulting image and quick access to recently opened files.

Its components include:

- **Live Preview:** Displays the cropped image using the required 800 × 650 aspect ratio.
- **Source Dimensions:** Displays the original image dimensions.
- **Crop Offset:** Displays the current horizontal and vertical crop coordinates.
- **Recent Files:** Displays up to six image thumbnails in a three-column grid.

- **Show All Button:** A small, pill-shaped button positioned beside the Recent Files heading. It becomes visible when more than six recent files are available.

This arrangement allows users to access recent images without leaving the main editing workspace.

**C. Right Panel – Main Editing Workspace**

The right panel contains the main image-editing canvas.

Its design includes:

- **Image Canvas:** Displays the selected image and its cropping boundary.
- **Crop Boundary:** Uses a lime-green outline to distinguish the selected cropping area.
- **Interactive Positioning:** Allows users to reposition the image through mouse interaction.
- **Close Session (×):** Positioned inside the upper-left area of the editing workspace rather than the main navigation bar.

The Gallery view was also designed to display images in a structured three-column grid.

Each gallery card was refined to maintain consistent dimensions, while long filenames were shortened using an ellipsis (...) to prevent text overflow.

An Exit button was positioned at the bottom-right of the Gallery view. Its floating placement allows the user to return to the editor without having to scroll back to the top.

## 2.3 System Architecture and Technology Stack

The application follows a Python-based desktop architecture.

| Component | Technology | Purpose |
|---|---|---|
| Programming Language | Python 3 | Core application logic |
| GUI Framework | CustomTkinter | Desktop interface and interactive controls |
| Image Processing | Pillow (PIL) | Image loading, cropping, resizing and exporting |
| Image Display | Tkinter Canvas / ImageTk | Image rendering and preview |
| Data Persistence | JSON | Storing recent files and activity statistics |
| Interface Design | Figma | UI/UX planning and visual design |

### 2.4 System Workflow

The application's primary workflow follows these stages:

1. The user launches Pro Cropper.
2. The application loads previously stored history and statistics.
3. The user selects an image through the Open Image button.
4. The image is displayed within the editing canvas.
5. The cropping engine calculates the appropriate cropping dimensions.
6. The user adjusts the image position through mouse interaction.
7. The left panel displays the updated live preview.
8. The user selects Save Image to export the processed result.
9. The application updates its saved statistics and history.
10. The user can access previously opened images through the Recent Files section or Gallery view.

---

# CHAPTER THREE: SYSTEM DEVELOPMENT AND IMPLEMENTATION

### 3.1 Development Environment

The application was implemented using Python 3, with CustomTkinter providing the graphical interface and Pillow handling the image-processing operations.

The development process involved translating the interface concepts designed in Figma into functional Python components.

Particular attention was given to ensuring that the visual elements were connected to actual application functionality rather than existing as decorative interface components.

### 3.2 Image Loading and Processing

The image-loading functionality was developed to support multiple image formats, including JPEG, PNG and WebP.

Pillow was used to open and decode the selected images. The implementation also incorporated EXIF orientation correction through `ImageOps.exif_transpose()` to ensure that images are displayed in their intended orientation.

Images are converted into RGB mode during the loading process to provide a consistent representation for subsequent processing.

## 3.3 Image Cropping Algorithm

One of the main technical components of Pro Cropper is its image-cropping engine.

The application uses a fixed output dimension of:

- Width: 800 pixels.
- Height: 650 pixels.
- Aspect ratio: 800:650.

A function named `best_crop_size()` calculates the largest possible rectangular cropping area that fits within the original image while maintaining the required aspect ratio.

The calculation follows this approach:

1. Determine the original image width and height.
2. Calculate the required cropping height based on the target aspect ratio.
3. If the calculated height exceeds the original image height, adjust the cropping width instead.
4. Return the largest valid cropping dimensions.

The `do_crop()` function then applies the selected cropping coordinates and resizes the resulting image to exactly 800 × 650 pixels using Pillow's high-quality LANCZOS resampling method.

This approach ensures that the image is cropped proportionally before resizing, rather than being stretched to fit the required dimensions.

## 3.4 Interactive Image Positioning

An interactive mouse-based positioning system was implemented using Tkinter event bindings.

The application uses mouse events such as:

- `<Button-1>` for mouse-click interaction.
- `<B1-Motion>` for continuous mouse movement while the button is held.

The system calculates the mouse position relative to the displayed image and converts the canvas coordinates into the original image coordinates.

The selected cropping area is then repositioned while ensuring that it remains within the original image boundaries.

This provides users with direct control over which part of the image appears in the final output.

## 3.5 Live Preview Implementation

A live preview component was developed to display the selected cropping result before export.

The application extracts the currently selected cropping area, resizes it for display and renders it inside the preview canvas.

The preview is updated when the image position changes, allowing users to evaluate their selection before saving.

The application also displays the original image dimensions and the current crop offsets.

## 3.6 Image Format Conversion and Export

One of the useful capabilities implemented in Pro Cropper is its support for opening WebP images and exporting the processed result as JPEG.

Pillow handles the image decoding, cropping and encoding processes.

The save functionality uses a file-dialog interface with JPEG as the default output format and a quality setting of 95 for JPEG exports.

This means users can process an image originally stored as WebP and produce a standard JPEG output without requiring a separate conversion application.

The implementation also supports other formats when the corresponding extension is selected in the save dialog.

## 3.7 Recent Files and Gallery Management

A recent-file management system was developed to improve accessibility and navigation.

The left panel displays up to six recent images using a three-column thumbnail grid.

When more than six images have been opened, a small Show All button becomes visible beside the Recent Files heading.

Selecting this button opens a dedicated Gallery view containing the complete recent-file collection.

The gallery was subsequently refined to improve its presentation:

- Cards maintain fixed dimensions for a consistent layout.
- Thumbnails use proportional cropping to prevent distortion.
- Long filenames are truncated and displayed with an ellipsis.
- Images are arranged in a three-column grid.
- A floating Exit button remains positioned at the bottom-right of the gallery interface.

These refinements improved visual consistency and navigation.

## 3.8 Application State and History Management

A JSON-based persistence mechanism was incorporated to maintain application information between sessions.

The `history.json` file stores relevant application information, including:

- Total edited images.
- Total saved images.
- Recently opened image file paths.

The application uses history-loading and history-saving functions to retrieve and update this information.

During application startup, previously stored image paths are checked to determine whether the files still exist.

Valid files are loaded to rebuild the thumbnail collection, while missing files are ignored to prevent invalid references from interrupting application execution.

This implementation enables the application to retain useful activity information without storing entire image files inside the JSON document.

## 3.9 Session Management

A session-management mechanism was implemented to protect users from accidentally losing unsaved work.

When the user selects the Close Session (✕) button while an image contains unsaved changes, the application displays a confirmation dialog.

The dialog provides the user with the opportunity to save changes, discard them or cancel the closing operation.

This improves the application's usability and reduces the possibility of accidental data loss.

## 3.10 Command-Line Processing

In addition to its graphical interface, the application includes a command-line processing mode.

This allows users to provide image files as input arguments and specify an output directory.

The command-line implementation also provides horizontal and vertical alignment options, including left, centre, right, top and bottom positioning.

This extends the application's functionality beyond interactive desktop editing and provides a foundation for batch image processing.

---

# CHAPTER FOUR: SYSTEM TESTING AND EVALUATION

## 4.1 Testing Approach

Testing was conducted to assess whether the application performed its intended operations correctly.

The testing process focused on image loading, cropping accuracy, format conversion, session management, gallery navigation and history integrity.

## 4.2 Functional Testing

The following tests were performed:

| Test Case | Testing Procedure | Expected Result | Observed Result |
|---|---|---|---|
| Image Loading | Open a supported image file. | Image should display in the editing canvas. | Successful |
| WebP Processing | Open a WebP image. | Image should load correctly. | Successful |
| Image Cropping | Apply the cropping operation. | Output should measure 800 × 650 pixels. | Successful |
| Image Positioning | Drag the image within the canvas. | Cropping area should update according to user positioning. | Successful |
| JPEG Export | Save a processed image as JPEG. | Image should export in JPEG format. | Successful |
| Unsaved Changes | Close a session before saving. | Confirmation dialog should appear. | Successful |
| Recent Files | Open multiple images. | Recent thumbnails should update. | Successful |
| Gallery Navigation | Select Show All when additional files exist. | Complete gallery should appear. | Implemented |

| | | | |
|---|---|---|---|
| Missing File Handling | Move a previously accessed image and restart the application. | Missing file should be ignored without crashing the application. | Successful |
| History Management | Restart the application after storing history. | Valid history and statistics should be restored. | Implemented |

## 4.3 Image Format Conversion Testing

Particular attention was given to testing the conversion of WebP images into JPEG output.

The application successfully opened WebP images through Pillow and allowed the resulting cropped image to be exported using the JPEG format.

The conversion process demonstrated the application's ability to work with different image formats within a single workflow.

## 4.4 Session and History Testing

Session management was evaluated by attempting to close an edited image before saving.

The application successfully displayed a confirmation message, allowing the user to decide whether to save, discard or cancel.

History integrity was also considered by moving a previously accessed image from its original location. The file-existence check was designed to ignore the missing file and prevent application failure.

## 4.5 User Interface Evaluation

The interface was refined throughout development to improve usability and visual consistency.

The main improvements included:

- Clear positioning of primary navigation buttons.
- Distinctive styling of the Save Image button.
- Improved organisation of recent-file thumbnails.
- A conditional Show All button.
- Uniform gallery card dimensions.
- Truncated filenames to prevent text overflow.
- A floating Gallery Exit button.

These refinements helped align the implemented application with the original interface design intentions established during the Figma phase.

### 4.6 Evaluation Summary

The development and testing activities demonstrated that Pro Cropper successfully implements its principal image-processing and desktop interaction requirements.

The application combines image manipulation, format conversion, gallery navigation and session management within a single desktop environment.

The project also demonstrates how interface design and software implementation can be integrated to produce a functional application based on clearly defined user requirements.

---

# CHAPTER FIVE: MAINTENANCE AND FUTURE ENHANCEMENTS

## 5.1 Current System Status

The application has reached a functional development stage, with its primary image-processing and interface components implemented.

Its current capabilities provide a foundation for additional improvements and future development.

## 5.2 Proposed Future Enhancements

The following improvements have been identified for subsequent development:

#### 1. Keyboard Shortcuts

Introduce keyboard shortcuts to improve accessibility and workflow efficiency.

- `Ctrl + S`: Activate the Save Image function.
- `Esc`: Close the active editing session or exit the current view.

#### 2. Image Zoom Functionality

Implement mouse scroll-wheel controls to allow users to zoom in and out of images within the editing canvas.

This would provide greater precision when positioning images and selecting specific cropping areas.

#### 3. Theme Customisation

Introduce a light/dark mode switch within the top navigation bar.

CustomTkinter's `set_appearance_mode()` functionality can be used to allow users to switch between available interface themes.

**4. Enhanced File Management**

Further improve recent-file management by introducing additional sorting and file organisation capabilities.

**5. Performance Optimisation**

Explore additional image-loading and memory-management improvements to support larger image collections and more demanding processing workflows.

---

# CONCLUSION

The development of Pro Cropper demonstrates the practical application of Software Engineering principles through the design and implementation of a functional desktop image-processing application.

The project followed the Software Development Life Cycle, beginning with requirements identification and interface planning in Figma, followed by system architecture, Python implementation, integration and testing.

The integration of Python 3, CustomTkinter, Pillow and JSON enabled the development of a desktop solution that combines image cropping, interactive positioning, live preview, image format conversion, recent-file management and application history.

One of the project's notable achievements is its ability to process WebP images and export them as high-quality JPEG files while maintaining a consistent 800 × 650 output dimension.

The project also highlights the importance of user-centred design, particularly through the deliberate positioning of interface controls, the organisation of the editing workspace and the refinement of gallery navigation.

Overall, Pro Cropper provides a practical demonstration of how software engineering concepts can be translated into a functional application that addresses a specific user need. Its modular architecture and proposed enhancements provide opportunities for continued development, improved usability and additional image-processing capabilities.

---

# REFERENCES

The following official technical resources provide supporting documentation for the technologies used in the project.

1. Python Software Foundation (n.d.) *Python 3 Documentation*. Available at: https://docs.python.org/3/
2. Pillow (n.d.) *Pillow Documentation*. Available at: https://pillow.readthedocs.io/
3. Tom Schimansky (n.d.) *CustomTkinter Documentation*. Available at: https://customtkinter.tomschimansky.com/
4. Figma (n.d.) *Figma: Design and Prototyping Resources*. Available at: https://www.figma.com/resource-library/

# APPENDICES

**Appendix A: User Interface Designs**

Include screenshots of the original Figma designs showing the planned navigation bar, preview panel, editing workspace and gallery layout.

**Appendix B: Application Screenshots**

Include screenshots demonstrating:

- Main application interface.
- Open Image and Save Image controls.
- Live image preview and cropping boundary.
- Recent Files section.
- Show All gallery.
- Floating Exit button.
- Image export dialog.

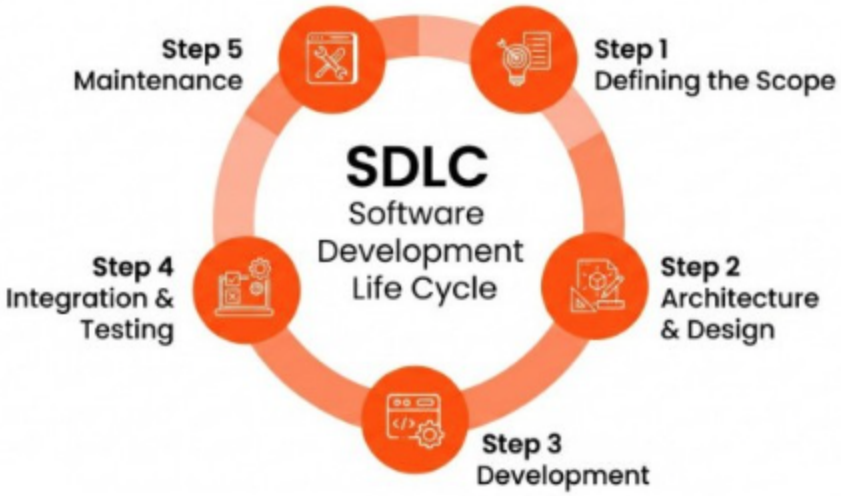
**Appendix C: Source Code**

Include the main Python source file (`pro_cropper.py`) and relevant supporting files.

Githb link -

**Appendix D: Application Data**

Include an example of the `history.json` file demonstrating how application statistics and recent image paths are stored.


Step 5
Maintenance
Step 1
Defining the Scope
SDLC
Software
Development
Life Cycle
Step 4
Integration &
Testing
Step 2
Architecture
& Design
Step 3
Development